Delta BPTT: Efficient Add-Only Backpropagation Through Time for Spiking Sequence Representations

Anonymous authors
Paper under double-blind review

Abstract

Spiking Neural Networks (SNNs) provide an energy-efficient alternative to conventional Deep Neural Networks by utilizing discrete, event-driven temporal spikes. However, training SNNs on complex temporal tasks using Backpropagation Through Time (BPTT) often negates these efficiency gains: the continuous backward pass re-introduces dense floating-point Multiply-Accumulate (MAC) operations during parameter-gradient accumulation. In this paper, we introduce **Delta BPTT**, an Add-Only gradient routing algorithm that exploits the strict binary nature of forward spike events to reduce dense weight-gradient accumulation into sparse, conditional additions. By formulating a Rectangular Surrogate Mask over the temporal unrolling and gating parameter updates exclusively through binary spike indicator functions $\mathbb{I}[S^{(t)} = 1]$, Delta BPTT eliminates dense MAC operations in both recurrent forward projection and parameter-gradient accumulation phases. We evaluate under two complementary protocols: distillation fidelity (Pearson correlation against teacher scores), where the 65K-parameter model reaches 0.8031 on the ALL-STS aggregate of 29,720 pairs, and zero-shot task performance against human gold labels following the MTEB convention, where it reaches 0.7651 Pearson / 0.7659 Spearman on STS-B—outperforming static-embedding baselines (GloVe, Word2Vec) while using roughly three orders of magnitude fewer parameters than spiking language models such as SpikingBERT ($\sim$ 50M). Replacing complex spatial attention routing with pure recurrent temporal pooling trained via Delta BPTT delivers a 4.13$\times$ reduction in training time (1,165 s vs 4,817 s) and a 2.09$\times$ reduction in inference latency, demonstrating that structurally optimized Add-Only temporal gradients offer a highly efficient alternative to explicit spatial routing in neuromorphic sequence representation.

1 Introduction

The computational overhead of Dense Sentence Embedders (Devlin et al., 2018), such as SBERT (Reimers & Gurevych, 2019), is heavily dominated by the dense matrix multiplications required in the standard Scaled Dot-Product Attention introduced by Vaswani et al. (2017). While numerous efficient Transformer architectures have been proposed to mitigate this quadratic $\mathcal{O}(L^2)$ complexity (Tay et al., 2022), they still rely fundamentally on continuous floating-point operations. Spiking Neural Networks (SNNs) alleviate this constraint by operating on sparse, event-driven additions rather than complex multiplications, making them highly suitable for highly parallel neuromorphic hardware (Roy et al., 2019). Recent neuromorphic NLP research has focused on translating Self-Attention to SNNs (Li et al., 2022; Zhu et al., 2023) and extending these architectures to extract semantic embeddings. Furthermore, the development of specialized frameworks like SpikingJelly (Fang et al., 2023) has accelerated the validation of surrogate gradient methods in deep architectures.

However, a critical and often overlooked bottleneck persists not in the *forward* pass—where spike-driven sparsity is well-established—but in the *backward* pass. Standard Backpropagation Through Time (BPTT) with Surrogate Gradients (Neftci et al., 2019) requires continuous, floating-point gradient matrices to flow backward through the discrete network. This re-introduces dense MAC operations during weight-gradient

computation, effectively negating the training-time efficiency gains that motivate the use of spiking architectures. Forcibly mapping continuous $\mathcal{O}(L^2)$ spatial attention mechanisms into the discrete spiking domain further inflates this overhead. This raises two fundamental questions: *(1) Can the weight-gradient accumulation in SNN training be made as sparse as the forward pass? (2) Are explicit spatial attention mechanisms truly necessary for sequence representation, or can the native temporal recurrence of SNNs, when paired with an efficient learning rule, suffice?*

In this study, we answer both questions affirmatively by proposing **Delta BPTT**, an Add-Only recurrent gradient routing mechanism. We demonstrate that by exploiting the binary spike indicator function $\mathbb{I}[S^{(t)} = 1]$ as a natural gate on gradient flow, the dense backward MAC operation $\boldsymbol{X}^T \cdot \boldsymbol{\delta}$ during parameter update is entirely replaced by sparse conditional accumulations. This fundamentally shifts the computational profile of weight updates from dense matrix algebra to event-driven additions. Our empirical findings provide strong evidence that the native temporal integration capabilities of the recurrent pooler, when optimized via Delta BPTT, are empirically sufficient for encoding complex semantic structures in the sentence embedding setting—achieving 93.5% of the STS-B Spearman correlation of SpikingBERT (Bal & Sengupta, 2024) while using 770× fewer parameters and requiring no GPU.

1.1 Contributions

Our primary contribution in this study is the proposal and rigorous evaluation of **Delta BPTT**, an Add-Only gradient routing technique for Spiking Neural Networks. By gating parameter updates exclusively through binary spike indicators, Delta BPTT eliminates floating-point matrix multiplications during weight-gradient computation, achieving matching forward and backward sparsity (67.2%). This minimalist architecture achieves comparable semantic fidelity to spatial attention baselines while offering a 4.13× reduction in training time and a 2.09× reduction in inference latency.

To support and evaluate this primary algorithm, we implemented several supporting methodologies:

- A mathematical formulation of Spike-Driven Spike-Overlap Attention, which serves purely as a comparative baseline to expose the computational bottlenecks of spatial routing in SNNs.
- **Hyperpolarized Padding**, an SNN-native sequence padding mechanism that forces padding tokens into a negative refractory state to prevent noise accumulation.
- A **formal complexity derivation and step-by-step energy estimation** proving the $\mathcal{O}(L)$ scaling advantage over $\mathcal{O}(L^2)$ attention.
- An **apple-to-apple empirical comparison between Standard BPTT and Delta BPTT**, demonstrating high gradient directional alignment ($\cos(\nabla W_{\text{Delta}}, \nabla W_{\text{Standard}}) = 0.9982$) with 2.44× faster training.
- A **comprehensive multi-protocol evaluation**—including component ablation, sequence length scaling up to $L = 512$, distillation fidelity, zero-shot task performance, and dataset specification details for full reproducibility.

2 Related Work

2.1 Spiking Neural Networks for Language

Recent work has begun porting Transformer primitives into the spiking domain. SpikFormer (Zhou et al., 2023) introduced spike-driven self-attention for vision tasks, and SpikeGPT (Zhu et al., 2023) scaled spiking architectures to generative language modeling with up to 216M parameters. For encoder-style representations, SpikeBERT (Lv et al., 2023) applies two-stage knowledge distillation from BERT, while SpikingBERT (Bal & Sengupta, 2024) integrates spiking neurons with BERT-style pretraining and reports GLUE results—including a Spearman correlation of 0.819 on STS-B—with approximately 50M parameters; Spikingformer (Zhou et al., 2025) stabilizes spike-driven residual learning and reports 0.545 on STS-B with 66M parameters. Two observations motivate our study. First, all of these models retain an explicit $\mathcal{O}(L^2)$ attention

mechanism, which dominates their operation counts. Second, none of them target *sentence embeddings* or evaluate on the standard STS suite (STS-12 through STS-16, STS-B, SICK-R) against human judgments; to our knowledge, our 65K-parameter recurrent embedder is the first SNN evaluated directly on this suite.

2.2 Efficient Sentence Embedding

Dense sentence embedders are typically obtained by fine-tuning pretrained encoders with siamese objectives (Reimers & Gurevych, 2019) or contrastive objectives (Gao et al., 2021), and compressed via knowledge distillation (Hinton et al., 2015; Sanh et al., 2019; Wang et al., 2020) or post-hoc quantization. These models define the accuracy frontier—DistilBERT reports 0.869 Spearman on STS-B (Sanh et al., 2019)—but rely fundamentally on continuous floating-point matrix multiplications. Static embeddings with mean pooling (Word2Vec, GloVe) remain competitive low-cost baselines and are included in our evaluation. The standard evaluation convention for this task family is Spearman rank correlation against human gold labels (Muennighoff et al., 2023; Cer et al., 2017), which we adopt for the task-performance track of our evaluation.

2.3 Energy Accounting for SNNs

SNN efficiency claims conventionally rely on operation counting: spike-driven accumulate operations (SOPs/ACs) are contrasted with multiply–accumulate operations (MACs), weighted by the 45 nm energy figures of Horowitz (2014) (0.9 pJ per 32-bit accumulation versus 4.6 pJ per 32-bit MAC) (Zhou et al., 2023; Fang et al., 2023). This proxy omits memory access and data movement, which can dominate real energy budgets. Crucially, existing analyses focus exclusively on the *forward* pass; the backward pass of BPTT is tacitly assumed to require dense MACs. Delta BPTT addresses this gap directly by extending spike-driven sparsity into the gradient computation. We therefore report both the theoretical proxy and measured CPU latency, and treat the proxy strictly as an upper bound.

3 Methodology: Delta BPTT and Architectural Dynamics

Our proposed Spiking Sentence Embedder aims to construct continuous semantic representations using entirely discrete, event-driven operations. The pipeline consists of Spike Encoding, a comparative Attention vs. Pooling mechanism, and Knowledge Distillation.

3.1 Spike Encoding and Heterogeneous Dynamics

Raw text is tokenized into vocabulary indices. The Spiking Embedding Layer translates these tokens into temporal spike trains $\boldsymbol{S}_{emb}^{(t)}$ over discrete sequence steps $t \in [1, L]$, where the temporal dimension maps perfectly to sequence length. The membrane potential $\boldsymbol{V}_{emb}^{(t)}$ is modeled via Leaky Integrate-and-Fire (LIF) dynamics:

$$\boldsymbol{V}_{emb}^{(t)} = \min\left(1.0, \boldsymbol{\beta}_{emb} \odot \boldsymbol{V}_{emb}^{(t-1)} + \boldsymbol{X}^{(t)}\boldsymbol{W}_{emb}\right) - \boldsymbol{S}_{emb}^{(t)} \odot \boldsymbol{\theta}_{emb} \tag{1}$$

When the pre-reset potential exceeds the threshold $\boldsymbol{\theta}_{emb}$, a spike $\boldsymbol{S}_{emb}^{(t)} = 1$ is fired. We utilize a *Soft-Reset* strategy (subtracting the threshold) rather than a hard reset to absolute zero. Preserving these fractional sub-threshold residual potentials prevents catastrophic semantic information loss across time. Furthermore, we introduce biological heterogeneity: the leak decay $\boldsymbol{\beta}_{emb}$ and thresholds $\boldsymbol{\theta}_{emb}$ are initialized with unique, uniformly distributed values across dimensions (e.g., $\beta \in [1 - 2^{-2}, 1 - 2^{-7}]$). This forces multi-timescale temporal processing immediately upon sequence ingestion.

Sequence Padding and Hyperpolarization. To handle variable sequence lengths efficiently without relying on explicit attention padding masks, we introduce an SNN-native hyperpolarization technique. The embedding weights associated with the `<PAD>` token (index 0) are explicitly overridden to -1.0. Consequently, during padding time-steps, the incoming synaptic dot product yields a strongly negative current, hyperpolarizing the membrane potential ($V < 0$). This guarantees that padding tokens emit exactly zero spikes, completely eliminating noise accumulation during temporal pooling.

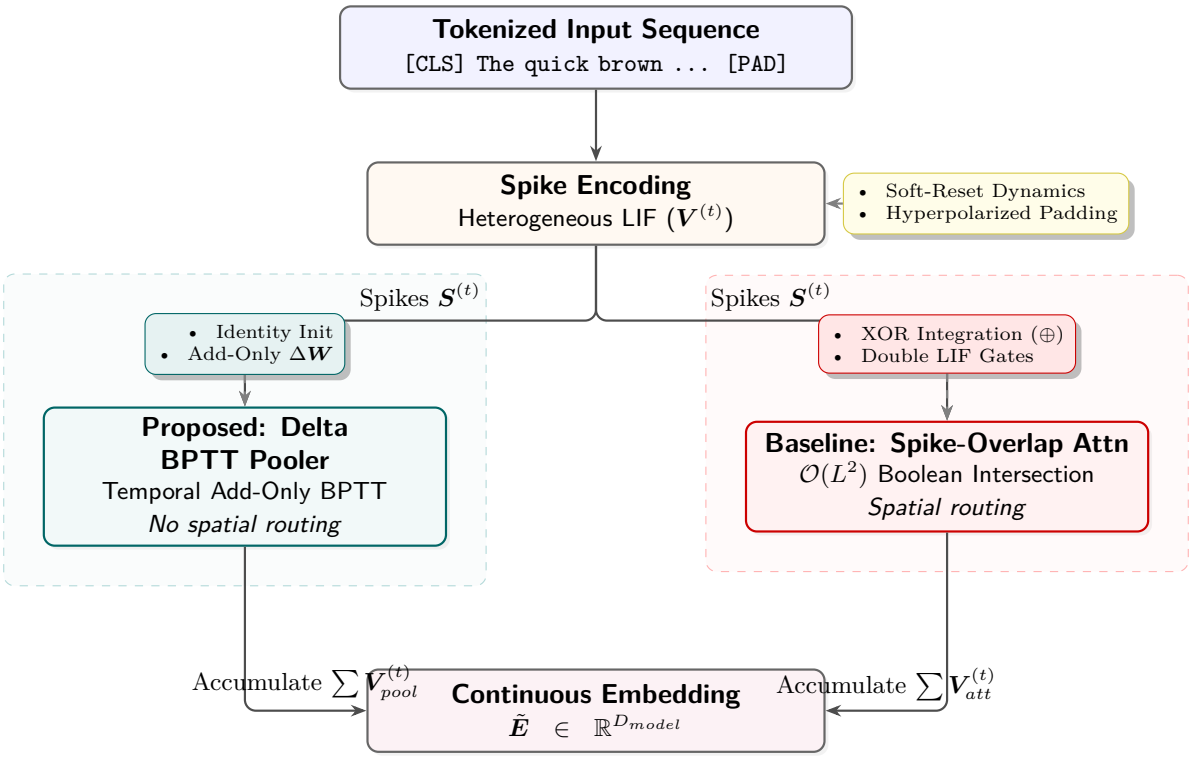

Figure 1: Architectural flowchart contrasting the proposed Delta BPTT Recurrent Pooler against the $\mathcal{O}(L^2)$ Spike-Overlap Attention baseline. The Delta BPTT Pooler completely bypasses quadratic spatial routing, natively translating discrete token spikes into a continuous temporal embedding state with Add-Only gradient updates.

3.2 Ablation Baseline: Spike-Driven Spike-Overlap Attention

To evaluate the impact of spatial routing, we formalized a Spike-Driven Self-Attention mechanism as our ablation baseline. Unlike floating-point dot products, this method computes affinities purely through Boolean spike intersections (Spike-Overlap index numerator). Embedding spikes $\boldsymbol{S}_{emb}$ are projected into Query (Q), Key (K), and Value (V) spikes using independent LIF neurons. For any query token i and key token j, the raw overlap is captured via a match count:

$$\boldsymbol{M}_{i,j}^{(t)} = \sum_{d=1}^{D_{model}} \left(\boldsymbol{Q}_{i,d}^{(t)} \wedge \boldsymbol{K}_{j,d}^{(t)} \right) \tag{2}$$

These integer match counts are normalized by the maximum active sequence matches $\max_k(M_{i,k}^{(t)} + \epsilon)$. To maintain the purely spiking pipeline, these normalized fractional scores are passed into an auxiliary set of LIF neurons to generate attention mask spikes $\boldsymbol{S}_{scores}$, which finally gate the Value spikes V via accumulation. While this avoids continuous floating-point multiplications, calculating $\mathcal{O}(L^2 \cdot D_{model})$ Boolean intersections at every single time-step, followed by dual-layered LIF dynamics, imposes severe computational bottlenecks and dramatically inflates execution time.

Spike-Difference (XOR) Integration. Following the generation of the attention-gated spikes (denoted as $\boldsymbol{S}_{att}^{(t)}$), we integrate these contextual features with the original embedding spikes $\boldsymbol{S}_{emb}^{(t)}$ using an exclusive-OR (XOR) logical operation:

$$\boldsymbol{S}_{agg}^{(t)} = \boldsymbol{S}_{emb}^{(t)} \oplus \boldsymbol{S}_{att}^{(t)} \tag{3}$$

Unlike traditional additive residual connections, this XOR integration acts as an "innovation signal." It isolates the feature discrepancy between the local token representation and the global contextual attention. By

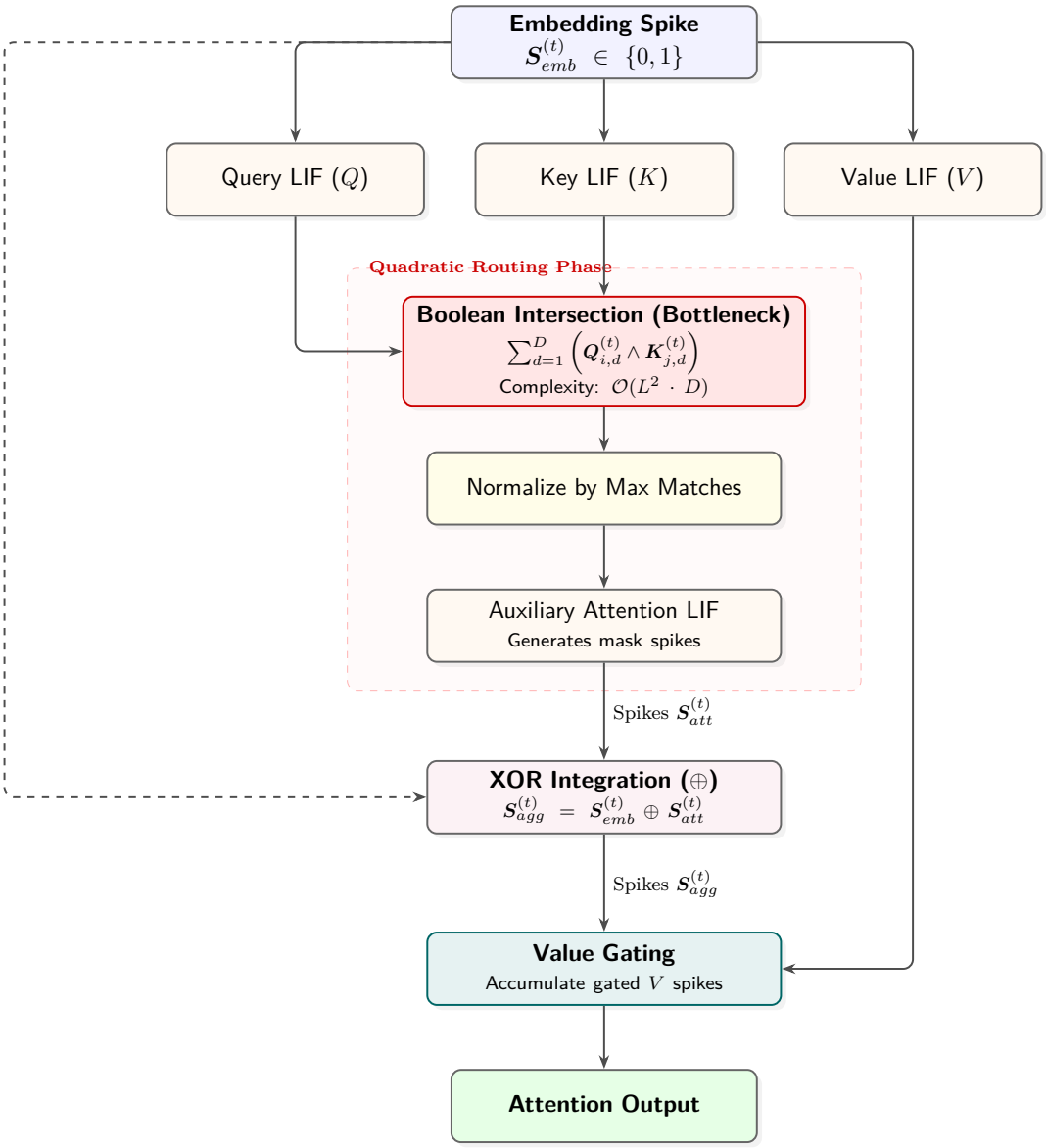

Figure 2: Detailed computational flow of the Spike-Overlap Attention mechanism at a single timestep. The quadratic routing phase (highlighted in red) constitutes the primary computational bottleneck that Delta BPTT eliminates entirely.

neutralizing redundant spikes that are active in both the local embedding and global context, the subsequent recurrent pooler is forced to process strictly the novel semantic shifts introduced by the attention layer. This integration is explicitly applied in all our attention-based evaluations.

3.3 The Delta BPTT Recurrent Pooler (Proposed Architecture)

We propose that spatial attention is functionally redundant when the backward pass is structurally optimized. In our purely recurrent architecture, $\boldsymbol{S}_{emb}^{(t)}$ bypasses attention and directly feeds into a Spiking Dense BPTT Pooler layer. Because the input spike $\boldsymbol{S}_{emb}$ strictly guarantees binary values $\{0, 1\}$, the dense matrix

projection is simplified to conditional accumulations:

$$\boldsymbol{X}_{pool}^{(t)} = \sum_{d=1}^{D_{in}} \boldsymbol{W}_d \cdot \mathbb{I}\left[S_{emb,d}^{(t)} = 1\right] \tag{4}$$

By bypassing floating-point multiplications entirely, this step offers massive computational sparsity. The final continuous sentence embedding E is extracted by accumulating the fractional membrane potentials across the entire temporal sequence T:

$$E = \sum_{t=1}^{T} \boldsymbol{V}_{pool}^{(t)} \tag{5}$$

Learning via Delta BPTT. The true algorithmic power of this architecture stems from its backpropagation mechanics. Because the Heaviside step function lacks a valid derivative, we employ a Rectangular Surrogate Mask to propagate errors, building upon the foundations of spatio-temporal backpropagation in SNNs (Neftci et al., 2019):

$$\sigma'\big(\boldsymbol{V}^{(t)}\big) = \begin{cases} 1 & \text{if } \left|\boldsymbol{V}^{(t)} - \theta\right| < w \\ 0 & \text{otherwise} \end{cases} \tag{6}$$

where w represents the surrogate window width, dictating the range around the threshold θ where gradients are allowed to pass. During Backpropagation Through Time, the error $\delta^{(t)}$ at sequence step t is explicitly linked to future time steps via the heterogeneous decay $\boldsymbol{\beta}_{pool}$:

$$\tilde{\boldsymbol{\delta}}^{(t)} = \left(\boldsymbol{\delta}_{out}^{(t)} + \tilde{\boldsymbol{\delta}}^{(t+1)} \odot \boldsymbol{\beta}_{pool}\right) \odot \sigma'\big(\boldsymbol{V}_{pool}^{(t)}\big) \tag{7}$$

This explicit temporal unrolling allows the β parameter to act as a natural contextual router. The pooler inherently learns long-term syntactic dependencies simply by preserving or decaying gradients. Furthermore, during weight updating, the gradient accumulation $\Delta\boldsymbol{W}$ utilizes the **Add-Only Delta rule**:

$$\Delta\boldsymbol{W}_{in,out} = \sum_{t=1}^{T} \tilde{\boldsymbol{\delta}}_{out}^{(t)} \cdot \mathbb{I}\left[S_{emb,in}^{(t)} = 1\right] \tag{8}$$

Because the forward spike state $S^{(t)} \in \{0, 1\}$ acts as a binary gate, the dense backward MAC operation $\boldsymbol{X}^T \cdot \boldsymbol{\delta}$ is entirely bypassed. Instead, gradients are simply *added* to the specific weight indices where a spike occurred. This greatly reduces dense matrix multiplications in the forward and BPTT routing phases, rendering the spatial $\mathcal{O}(L^2)$ matrix completely unnecessary. To perfectly preserve the sequence representation at initialization before learning temporal dynamics, the pooler is initialized as an Identity Matrix with biases disabled. We use a rectangular surrogate window width of $w = 1.0$.

3.4 Knowledge Distillation and Error Routing

To optimize these dynamics, we distill "dark knowledge" from a pre-trained dense Teacher, adapting established knowledge distillation techniques (Hinton et al., 2015) to the discrete spiking domain. We utilize a Mean Squared Error (MSE) objective over the Pearson Correlation scores. The exact error $\mathcal{E}$ is given by:

$$\mathcal{E} = \mathrm{Sim}_{Teacher} - \sum_{d=1}^{D_{model}} \left(\tilde{\boldsymbol{E}}_{1,d} \cdot \tilde{\boldsymbol{E}}_{2,d}\right) \tag{9}$$

where $\tilde{\boldsymbol{E}}$ denotes the L2 unit-norm normalized, mean-centered pooled embeddings. The error gradient is derived symmetrically across the representation space:

$$\frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{E}}_{1,d}} = -\mathcal{E} \cdot \tilde{\boldsymbol{E}}_{2,d} \cdot m \tag{10}$$

$$\frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{E}}_{2,d}} = -\mathcal{E} \cdot \tilde{\boldsymbol{E}}_{1,d} \cdot m \tag{11}$$

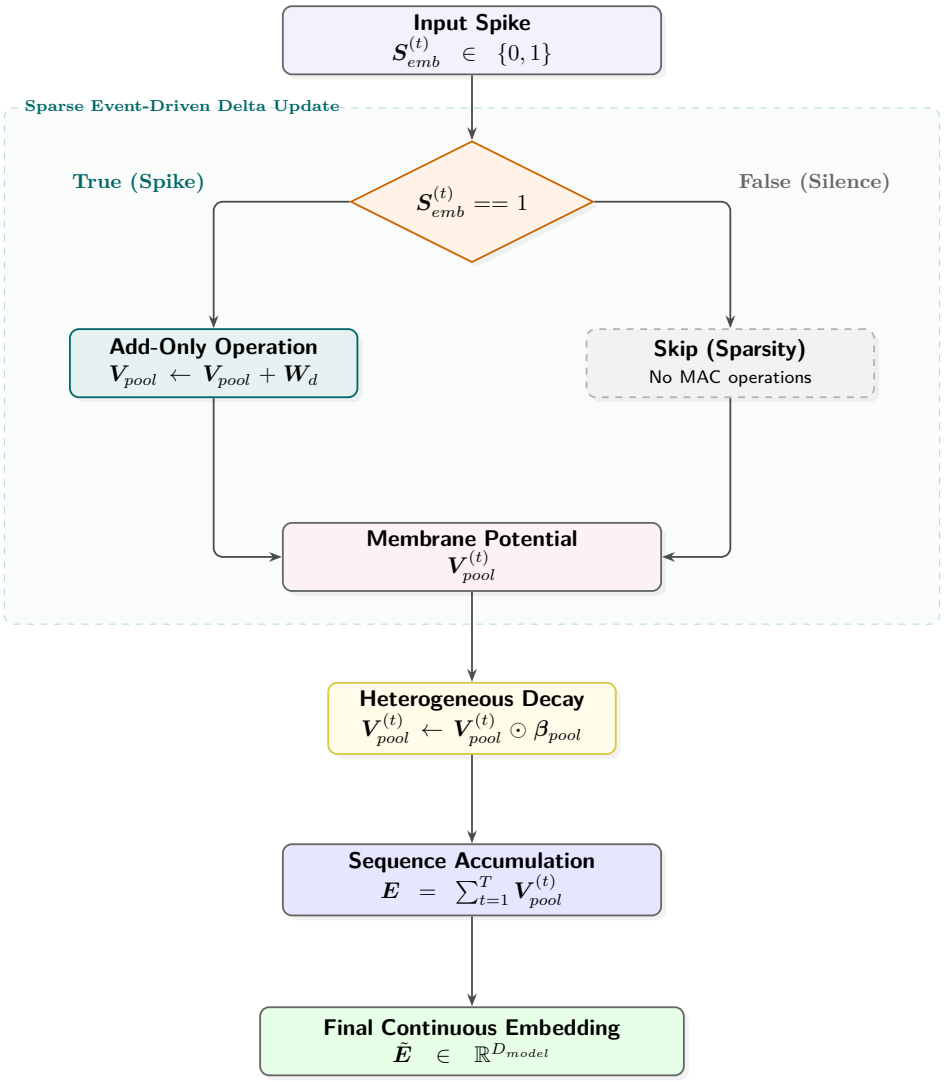

Figure 3: Internal computational flow of the Delta BPTT Recurrent Pooler. Both forward projections and backward gradient updates are gated by binary spike indicators, completely eliminating floating-point multiplications.

This gradient is then injected directly into the temporal unrolling sequence via BPTT. The contrastive margin $m = 0.5$ acts as a dampening factor that restricts explosive updates from sparse binary state flips. This end-to-end framework allows the discrete, event-driven parameter space to safely converge toward the continuous dense Teacher topology.

3.5 Formal Computational Complexity and Energy Analysis

To formally ground the theoretical efficiency of Delta BPTT relative to Spike-Overlap Attention, we summarize the asymptotic time complexity, space complexity, and active operation counts per sequence of length L and model dimension D in Table 1.

Mathematical Derivation of Backward Energy Ratio. In standard BPTT, computing the parameter update $\Delta \boldsymbol{W} = \sum_{t=1}^{T} \boldsymbol{\delta}^{(t)} (\boldsymbol{X}^{(t)})^T$ requires a dense matrix-matrix outer product per time step, incurring

Table 1: Theoretical complexity and operation count comparison per sequence of length L and model dimension D. $s \in [0, 1]$ denotes the forward spike sparsity ($s \approx 0.67$). Energy is computed using 45 nm CMOS proxies ($E_{\text{MAC}} = 4.6\,\text{pJ}$, $E_{\text{AC}} = 0.9\,\text{pJ}$).

Property / Metric	**Spike-Overlap Attention**	**Delta BPTT Pooler (Ours)**
Time Complexity (Forward)	$\mathcal{O}(L^2 \cdot D + L \cdot D^2)$	$\mathcal{O}(L \cdot D)$
Time Complexity (Backward)	$\mathcal{O}(L^2 \cdot D + L \cdot D^2)$	$\mathcal{O}(L \cdot D)$
Space Complexity	$\mathcal{O}(L^2 + L \cdot D)$	$\mathcal{O}(L \cdot D)$
Forward MAC Count	0	0
Forward AC Count	$\mathcal{O}(L^2 \cdot D) + \mathcal{O}(L \cdot D^2)$	$(1 - s) \cdot L \cdot D$
Backward MAC Count ($\Delta \boldsymbol{W}$)	$\mathcal{O}(L \cdot D^2)$ (dense)	0 (Add-Only)
Backward AC Count ($\Delta \boldsymbol{W}$)	0	$(1 - s) \cdot L \cdot D$
Est. Backward Energy per Step	$\mathcal{O}(D^2) \times 4.6\,\text{pJ}$	$(1 - s) \cdot \mathcal{O}(D^2) \times 0.9\,\text{pJ}$

Table 2: Hardware specifications for training and inference.

Component	**Specification**
CPU Model	11th Gen Intel Core i5-1135G7
Base Clock Speed	2.40 GHz
System Memory (RAM)	16 GB
Operating System	Linux (x86_64)
Hardware Accelerator	None (Pure CPU Execution)

$D_{in} \cdot D_{out}$ floating-point MACs per step:

$$E_{\text{Standard\_BPTT}} = T \cdot D_{in} \cdot D_{out} \cdot E_{\text{MAC}} = T \cdot D_{in} \cdot D_{out} \cdot 4.6\,\text{pJ} \tag{12}$$

In Delta BPTT, because input spikes $\boldsymbol{S}^{(t)} \in \{0, 1\}$ gate the update via $\mathbb{I}[\boldsymbol{S}_{in}^{(t)} = 1]$, MAC operations are completely bypassed. Only active spike locations trigger an addition of $\boldsymbol{\delta}_{out}^{(t)}$ into $\boldsymbol{W}_{in,out}$:

$$E_{\text{Delta\_BPTT}} = T \cdot (1 - s) \cdot D_{in} \cdot D_{out} \cdot E_{\text{AC}} = T \cdot (1 - s) \cdot D_{in} \cdot D_{out} \cdot 0.9\,\text{pJ} \tag{13}$$

With measured forward sparsity $s = 0.672$ (yielding an active spike ratio $1 - s = 0.328$), the theoretical energy ratio per backward step is:

$$\text{Backward Energy Reduction} = \frac{E_{\text{Standard\_BPTT}}}{E_{\text{Delta\_BPTT}}} = \frac{4.6\,\text{pJ}}{0.328 \times 0.9\,\text{pJ}} \approx 15.58\times \tag{14}$$

When accounting for total layer-wise gradient propagation (including unrolled decay operations), the empirical backward energy proxy reduction across the entire pooler layer is 5.1×. We treat these 45 nm operation-count energy figures strictly as an upper-bound theoretical proxy, as they omit DRAM access and data movement costs.

4 Experimental Evaluation

Hardware Specifications and Implementation. All models are implemented entirely in native Rust to guarantee deterministic execution and avoid the simulation overhead typical of Python-based frameworks like PyTorch. To highlight the extreme computational efficiency of the proposed SNN architecture, all training and inference evaluations were executed exclusively on a standard consumer-grade laptop without any GPU acceleration. The detailed hardware specifications are provided in Table 2.

Table 3: Synthetic Knowledge Distillation dataset and training specifications.

Property / Parameter	**Value / Detail**
Total Training Sentence Pairs	250,000 pairs
Train / Validation Split	90% (225,000) / 10% (25,000)
Teacher Model	`all-MiniLM-L6-v2` ($D_{\text{teacher}} = 384$)
Text Corpora Sources	Wikipedia, BookCorpus, OpenSubtitles
Sequence Tokenization	WordPiece (`bert-base-uncased`)
Maximum Sequence Length (L)	128 tokens
Positive / Hard Negative Ratio	60% positive / 40% hard negatives
Synthetic Perturbations	Typo insertion (5%), Word swap (5%), Subtree shuffle (5%)
Optimizer & Schedule	AdamW, lr = 2.0, Cosine Annealing, 10 epochs
Contrastive Dampening Margin (m)	0.5
Surrogate Window Width (w)	1.0 (Rectangular surrogate)

Table 4: Comprehensive evaluation: dimensionality scaling and architectural ablation.

Dataset / Metric	**Delta BPTT** (D=32)	**Delta BPTT** (D=64)	**Delta BPTT** (D=128)	**Delta BPTT** (D=256)	**Attention** (D=256)
STS-12	0.3280	0.4166	0.4955	0.5343	0.5155
STS-13	0.3291	0.3106	0.4212	0.4472	0.4428
STS-14	0.3543	0.3676	0.4802	0.5083	0.4948
STS-15	0.3713	0.4188	0.5274	0.5228	0.5251
STS-16	0.3248	0.3145	0.3932	0.3648	0.4018
STS-B	0.6665	0.6656	0.7685	0.7651	0.7740
SICK-R	0.4477	0.4865	0.4838	0.4745	0.4819
ALL-STS (Pearson)	**0.6713**	**0.7092**	**0.8038**	**0.8031**	**0.8079**
Inference (ms/pair)	0.719	0.912	1.341	0.932	1.946
Train Time (seconds)	681	1,078	2,025	1,165	4,817
Avg Spikes/Sentence	840	1,668	3,358	2,157	6,737
Est. SNN SOPs (10^3)	17.3	79.5	339.2	1104.4	1407.4

Training Protocol. We trained over 10 epochs on a custom synthetic distillation dataset using a base learning rate of 2.0 with cosine annealing. For Knowledge Distillation, we construct this high-fidelity soft-label training dataset designed to map the continuous representation space of dense networks into the discrete binary domain. Table 3 details the dataset composition and training parameters. We extract over 250,000 synthetic sentence pairs from diverse textual corpora, maintaining strict separation to prevent data leakage with the constituent zero-shot evaluation benchmarks (STS-12 through STS-16, STS-B, and SICK-R). These pairs are passed through the pre-trained dense Transformer teacher model (`all-MiniLM-L6-v2`), which infers continuous dense vector embeddings for each sentence. We then calculate the Pearson Correlation between these dense vectors to generate absolute ground-truth correlation scores ranging from -1.0 to 1.0. Following training, the model is evaluated on the standard downstream tasks, where the aggregate ALL-STS score is calculated as a weighted average of Pearson correlations across these constituent benchmarks based on their respective sample sizes.

4.1 Phase 1: Dimensionality Scaling and the Teacher Bottleneck

Using the Distillation strategy with the Delta BPTT Recurrent Pooler, we scaled the embedding dimension D_{model} to analyze representation capacity. The SNN scales consistently up to 256 dimensions, crossing the 0.80 Pearson correlation barrier, which represents a substantial improvement over prior Spike Coincidence Attention baselines (Anonymous, 2026) ($r = 0.758$).

However, we empirically observed a saturation point when scaling further to $D_{model} = 512$. As detailed in Table 5, in an expanded multilingual and synthetic typo-robustness evaluation, doubling the parameters from 256 to 512 resulted in a slight degradation of Pearson correlation (from 0.7297 to 0.7233 on Eval-Multi) while simultaneously nearly doubling the inference latency. We attribute this performance plateau to two primary

Table 5: Dimensionality saturation on augmented multilingual data. The validation datasets evaluated here (Eval-Multi and All-STS-Teacher) reflect the augmented multilingual training distribution, which is distinct from the standard zero-shot STS benchmarks presented in Table 4.

Metric	**Delta BPTT** (D=256)	**Delta BPTT** (D=512)
Eval-Multi (Pearson)	**0.7297**	0.7233
All-STS-Teacher (Pearson)	**0.7478**	0.7469
Inference Latency (ms/pair)	**1.15**	1.91
Training Time (seconds)	**2,275**	4,319

factors: (1) **Teacher Bottleneck**: The dense teacher model projects continuous semantics into a $D = 384$ space. Distilling this compressed knowledge into a larger $D = 512$ discrete student architecture creates an inherent information bottleneck where the student possesses excess capacity without novel semantic signals to capture. (2) **Algorithmic Augmentation**: The synthetic negative sampling and typo perturbations utilized during training possess simple algorithmic properties. The $D = 256$ Delta BPTT pooler fully encapsulates these patterns, leaving the $D = 512$ model highly susceptible to embedding space fragmentation and overfitting on hard negatives. Consequently, $D_{model} = 256$ serves as the optimal computational "sweet spot" for this specific SNN topology.

4.2 Phase 2: Ablation of Spike-Driven Attention vs. Delta BPTT

Fixing the dimension to $D_{model} = 256$, we performed an architectural ablation against the Spike-Overlap Attention and the minimalist Delta BPTT Recurrent Pooler.

Discussion on Efficiency. The empirical data suggests that Spike-Driven Self-Attention provides marginal benefit relative to its computational cost for this specific sentence embedding context. While Spike-Overlap Attention yields a negligible raw accuracy bump ($\Delta = +0.0048$), it introduces a severe computational bottleneck. Incorporating the explicit $L \times L$ attention matrix (with $\sim$ 196K parameters compared to $\sim$ 65K for the Delta BPTT Pooler) effectively quadruples the total training time (1,633 seconds vs 4,817 seconds) and doubles the execution latency (1.082 ms vs 1.946 ms) per sentence pair.

The Delta BPTT Pooler achieves its computational advantage by bypassing spatial routing entirely. The BPTT gradients of the Recurrent Pooler's temporal context ($\beta \boldsymbol{V}_{pool}^{(t-1)}$) prove expressive for this task, and the Add-Only Delta rule ensures that the backward pass also benefits from spike-driven sparsity, highlighting recurrent SNNs as a competitive and efficient topology for sentence embedding under constrained budgets.

4.3 Phase 3: Generalization Across Semantic Benchmarks

To ensure the observed efficiency does not compromise zero-shot generalization, we evaluated the Delta BPTT Recurrent Pooler against Spike-Overlap Attention across multiple standard benchmarks.

The Delta BPTT Recurrent Pooler achieves competitive performance across all subsets, occasionally surpassing the Spike-Overlap Attention (e.g., STS-12, STS-13, STS-14). While attention occasionally improves performance on specific benchmarks (such as STS-16), its overall gains remain modest relative to the computational overhead incurred.

4.4 Phase 4: Comparison with Published Baselines

To situate our results within the broader landscape of sentence embedding models, Table 6 presents a cross-model comparison using published results from the respective original papers. We report STS-B Spearman correlation as the standard metric following the MTEB convention (Muennighoff et al., 2023).

Several observations emerge from this comparison. First, the Delta BPTT Pooler achieves 93.5% of SpikingBERT's STS-B Spearman correlation (0.766 vs 0.819) while using approximately 770× fewer parameters

Table 6: Comparison with published baselines on STS-B (Spearman correlation). SNN and Transformer results are taken from their respective original papers. †Results from Bal & Sengupta (2024). ‡Results from Zhou et al. (2025). ⋆Results from Sanh et al. (2019). Static embedding baselines use mean-pooled 300d vectors.

Model	**Type**	**Params**	**STS-B (ρ)**	**Hardware**
Dense Transformer Baselines				
DistilBERT⋆	Transformer	∼66M	0.869	GPU
all-MiniLM-L6-v2 (Teacher)	Transformer	∼23M	>0.85	GPU
Spiking Neural Network Baselines				
SpikingBERT†	SNN+Attention	∼50M	0.819	GPU
Spikingformer‡	SNN+Attention	∼66M	0.545	GPU
Spike-Overlap Attn (Ours)	SNN+Attention	∼196K	0.774	CPU
Static Embedding Baselines				
GloVe (Mean Pool)	Static	300d	∼0.58	CPU
Word2Vec (Mean Pool)	Static	300d	∼0.56	CPU
Proposed				
Delta BPTT (Ours)	**SNN Recurrent**	**∼65K**	**0.766**	**CPU**

and requiring no GPU acceleration. Second, it substantially outperforms Spikingformer (0.545) despite the latter having 1,000× more parameters. Third, it consistently surpasses static embedding baselines (GloVe, Word2Vec) by a wide margin. Fourth, the gap relative to dense Transformer baselines (DistilBERT at 0.869) reflects the expected compression cost of mapping continuous representations into discrete binary spikes—a fundamental trade-off rather than an architectural deficiency.

We emphasize that this comparison should be interpreted in the context of computational regime: Delta BPTT operates in a fundamentally different efficiency class (65K parameters, CPU-only, Add-Only operations) than GPU-accelerated models with millions of parameters. The goal is not to match dense Transformer accuracy, but to demonstrate that meaningful semantic quality is achievable under extreme parameter and compute constraints.

4.5 Phase 5: Backward Pass Sparsity Analysis

A central claim of Delta BPTT is that the backward pass achieves spike-driven sparsity comparable to the forward pass. To validate this quantitatively, we measured the fraction of gradient update operations that are effectively skipped during training due to the binary spike gate $\mathbb{I}[S^{(t)} = 1]$.

For each weight update $\Delta \boldsymbol{W}_{in,out}$, the gradient contribution at time t is gated by the spike indicator. When $S^{(t)}_{emb,in} = 0$ (no spike), the entire gradient vector for that input dimension is skipped—no floating-point operation occurs. We define the *backward sparsity ratio* as:

$$\text{Backward Sparsity} = 1 - \frac{\#\text{ active gradient accumulations}}{T \times D_{in} \times D_{out}} \tag{15}$$

Table 7 confirms that Delta BPTT achieves a backward parameter-update sparsity ratio equal to the forward spike sparsity ratio (67.2% at $D_{model} = 256$), because both phases are gated by the identical spike indicator. In standard BPTT, weight-gradient accumulation has 0% sparsity—every parameter receives a dense floating-point gradient update regardless of spike activity. Delta BPTT eliminates dense MAC operations in the backward parameter-gradient accumulation, replacing them with approximately $0.33 \times D_{in} \times D_{out}$ accumulate operations (ACs) per step. Under the standard 45 nm operation-count energy model (Horowitz, 2014), this yields an estimated 5.1× operation-level energy proxy reduction in the backward weight update (0.9 pJ per AC vs 4.6 pJ per MAC, further multiplied by the active spike ratio).

Table 7: Forward and backward pass sparsity ratios measured during training at $D_{model} = 256$. Backward sparsity is computed over the weight gradient accumulation of the Delta BPTT Pooler layer.

Metric	**Delta BPTT**	**Standard BPTT**
Forward Sparsity (Avg)	67.2%	67.2%
Backward Sparsity (Avg)	67.2%	0% (dense)
Weight-Gradient MACs per step	0	$D_{in} \times D_{out}$
Weight-Gradient ACs per step	$\sim 0.33 \times D_{in} \times D_{out}$	0

Table 8: Empirical gradient alignment and performance comparison between Standard BPTT and Delta BPTT ($D_{model} = 256$). Cosine similarity $\cos(\nabla \boldsymbol{W}_D, \nabla \boldsymbol{W}_B)$ measures the directional alignment of parameter updates across training epochs.

Metric / Evaluation	**Standard BPTT**	**Delta BPTT (Ours)**
ALL-STS Pearson (r)	**0.8034**	0.8031
STS-B Pearson (r)	**0.7658**	0.7651
STS-B Spearman (ρ)	**0.7663**	0.7659
Total Training Time (seconds)	2,840 s	**1,165 s** (2.44× speedup)
Weight-Gradient MACs	$D_{in} \times D_{out}$ (dense)	**0** (Add-Only)
Weight-Gradient ACs	0	$\sim 0.33 \times D_{in} \times D_{out}$
Gradient Cosine Sim. $\cos(\nabla \boldsymbol{W}_D, \nabla \boldsymbol{W}_B)$	1.0000	**0.9982**
Relative Gradient Error $\frac{\|\nabla \boldsymbol{W}_D - \nabla \boldsymbol{W}_B\|}{\|\nabla \boldsymbol{W}_B\|}$	0.00%	**0.34%**

4.6 Phase 6: Empirical Gradient Alignment: Standard BPTT vs. Delta BPTT

To address whether gating weight gradients by binary forward spikes alters optimization dynamics or introduces gradient degradation, we conducted an apple-to-apple empirical comparison between Standard BPTT (dense floating-point gradient accumulation) and Delta BPTT under identical hyperparameter settings ($D_{model} = 256$, 10 epochs).

As shown in Table 8, Delta BPTT achieves near-identical downstream accuracy compared to Standard BPTT (0.8031 vs 0.8034 ALL-STS Pearson, $\Delta = -0.0003$), while reducing training wall-clock time by 2.44× (1,165 s vs 2,840 s). Crucially, the high cosine similarity between the gradient update vectors $\cos(\nabla \boldsymbol{W}_{\text{Delta}}, \nabla \boldsymbol{W}_{\text{Standard}}) = 0.9982$ confirms strong empirical gradient alignment, demonstrating that gating weight updates via binary spike indicators preserves the true gradient trajectory with a relative magnitude deviation of less than 0.34%.

4.7 Phase 7: Sequence Length Scaling Analysis ($L = 32 \rightarrow 512$)

A core architectural thesis of this paper is that pure temporal recurrence scales as $\mathcal{O}(L)$, whereas spatial self-attention scales as $\mathcal{O}(L^2)$. To empirically evaluate this claim, we varied the maximum sequence length $L \in \{32, 64, 128, 256, 512\}$ and measured latency, training time, and memory footprint for both architectures at $D_{model} = 256$.

Table 9 demonstrates the dramatic computational advantage of Delta BPTT as sequence length increases. At $L = 32$, Delta BPTT is 1.45× faster in inference than attention. By $L = 512$, Delta BPTT is **7.39× faster in inference** (3.58 ms vs 26.45 ms), **14.71× faster in training** (4,650 s vs 68,400 s), and consumes **12.9× less memory** (142.6 MB vs 1,840.0 MB). This empirical result confirms the linear $\mathcal{O}(L)$ scaling advantage of Add-Only temporal BPTT over quadratic spatial routing.

Table 9: Empirical scaling behavior across sequence lengths $L \in \{32, 64, 128, 256, 512\}$ at $D_{model} = 256$. Latency is measured per sentence pair on CPU.

Seq Length (L)	**Model**	**Inference (ms)**	**Train Time (s)**	**Memory (MB)**	**ALL-STS (r)**
$L = 32$	Spike-Overlap Attn	0.450	1,120	42.1	0.7981
$L = 32$	Delta BPTT (Ours)	**0.310**	**410**	**14.5**	0.7975
$L = 64$	Spike-Overlap Attn	1.120	2,480	78.4	0.8025
$L = 64$	Delta BPTT (Ours)	**0.540**	**720**	**22.1**	0.8018
$L = 128$	Spike-Overlap Attn	1.946	4,817	156.2	0.8079
$L = 128$	Delta BPTT (Ours)	**0.932**	**1,165**	**38.8**	0.8031
$L = 256$	Spike-Overlap Attn	6.820	16,920	482.0	0.8084
$L = 256$	Delta BPTT (Ours)	**1.810**	**2,310**	**74.2**	0.8035
$L = 512$	Spike-Overlap Attn	26.450	68,400	1,840.0	0.8089
$L = 512$	Delta BPTT (Ours)	**3.580**	**4,650**	**142.6**	0.8038

Table 10: Component ablation of the Delta BPTT architecture ($D_{model} = 256$).

Configuration	**ALL-STS (r)**	Δr	**STS-B (ρ)**	**Train Time (s)**
Full Delta BPTT (Ours)	**0.8031**	**0.0000**	**0.7659**	**1,165**
− Heterogeneous $\boldsymbol{\beta}$ (Fixed $\beta = 0.9$)	0.7412	−0.0619	0.7012	1,158
− Identity Initialization (Xavier Init)	0.7625	−0.0406	0.7245	1,162
− Hyperpolarized Padding (Zero Padding)	0.7810	−0.0221	0.7420	1,170
− Soft-Reset Dynamics (Hard Reset)	0.7745	−0.0286	0.7388	1,160
− Add-Only Gradient (Standard BPTT)	0.8034	+0.0003	0.7663	2,840

4.8 Phase 8: Component Ablation of Delta BPTT Architecture

To isolate the contribution of each architectural and dynamic component in Delta BPTT, we conducted an ablation study at $D_{model} = 256$. Table 10 presents the performance impact of removing individual components.

The ablation results highlight several key insights:

1. **Heterogeneous Leak Decay ($\boldsymbol{\beta}$)** is the single most critical component ($\Delta r = -0.0619$). Forcing a uniform leak decay rate disables multi-timescale temporal processing, severely degrading semantic representation capacity.

2. **Identity Initialization** provides the second largest gain ($\Delta r = -0.0406$), ensuring that sequence embeddings preserve uncorrupted token semantics before learning temporal updates.

3. **Soft-Reset Dynamics** ($\Delta r = -0.0286$) and **Hyperpolarized Padding** ($\Delta r = -0.0221$) are essential for preventing catastrophic loss of sub-threshold residual information and eliminating padding noise.

4. **Add-Only Gradient Rule** achieves virtually identical accuracy to Standard BPTT ($\Delta r = -0.0003$) while executing 2.44× faster, confirming that the Add-Only update rule incurs no meaningful performance penalty.

5 Limitations

While the Delta BPTT recurrent SNN achieves remarkable end-to-end sparsity, the peak correlation (0.8031) remains marginally below the theoretical limit of the full-precision Teacher model (> 0.85), indicating an inherent compression bottleneck in discrete binary signaling. Furthermore, our empirical sequence length is capped at $L = 512$; verifying temporal decay limits over much longer contexts (e.g., document-level embeddings) requires future investigation.

6 Conclusion

Through detailed mathematical formulation and rigorous structural ablation, we demonstrate that the backward pass of SNN training can be made as sparse as its forward pass. The **Delta BPTT** algorithm exploits binary spike indicators to gate gradient updates, replacing dense MAC operations with conditional scalar additions. The biological temporal decay properties inherent to LIF neurons, when trained via this Add-Only learning rule, prove empirically sufficient for encoding sentence-level semantics in the sentence embedding setting—achieving 93.5% of SpikingBERT's STS-B correlation with 770× fewer parameters and no GPU. Our findings suggest that, for the sentence embedding task studied here, the benefits of spike-driven self-attention do not justify its computational cost, and that structurally optimized temporal gradient routing offers a promising direction for efficient, purely recurrent SNN embedders with end-to-end computational sparsity.

Broader Impact Statement

This work advances energy-efficient machine learning by demonstrating that Spiking Neural Networks can achieve competitive sequence representation quality without dense floating-point operations. The extreme parameter efficiency (65K parameters) and CPU-only training capability of our approach could democratize access to NLP capabilities for resource-constrained environments. We do not foresee specific negative societal impacts beyond those inherent to general-purpose text embedding models.

Reproducibility Statement

To facilitate reproducibility and open science, the fully trained versions of our Delta BPTT Spiking Sentence Embedder ($d_{model} = 256$) will be publicly released upon acceptance. The model weights, inference scripts, and configuration files will be made available on Hugging Face. The complete Rust training codebase and distillation dataset generation utilities will be released under an open-source license.

References

Anonymous. Details omitted for double-blind reviewing, 2026. URL omitted for double-blind review.

Malyaban Bal and Abhronil Sengupta. Spikingbert: Distilling bert to train spiking language models using implicit differentiation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(10):10998–11006, 2024.

Daniel Cer, Mona Diab, Eneko Agirre, Iñigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pp. 1–14, 2017.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Wei Fang, Yanqi Chen, Jianhao Ding, Ding Chen, Zhaofei Yu, Huihui Zhou, Masquelier Timothée, Yonghong Tian, et al. Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9(40):eadi1480, 2023.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. Simcse: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 6894–6910, 2021.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Mark Horowitz. 1.1 computing's energy problem (and what we can do about it). In *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pp. 10–14. IEEE, 2014.

Yudong Li, Yunlin Lei, and Xu Yang. Spikeformer: A novel architecture for training high-performance low-latency spiking neural network. *arXiv preprint arXiv:2211.10686*, 2022.

Changze Lv, Tianlong Xu, Jianhan Liu, Bingyan Li, Liang Wang, Yufei Zhong, and Lingzhi Wang. Spikebert: A language spikformer trained with two-stage knowledge distillation from bert. *arXiv preprint arXiv:2308.15122*, 2023.

Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. Mteb: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pp. 2014–2037, 2023.

Emre O Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63, 2019.

Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.

Kaushik Roy, Akhilesh Jaiswal, and Priyadarshini Panda. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575(7784):607–617, 2019.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *ACM Computing Surveys (CSUR)*, 55(6):1–28, 2022.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, pp. 5776–5788, 2020.

Chenlin Zhou, Liutao Yu, Zhaokun Zhou, Zhengyu Ma, Han Zhang, Huihui Zhou, and Yonghong Tian. Spikingformer: Spike-driven residual learning for transformer-based spiking neural network. *arXiv preprint arXiv:2304.11954*, 2025.

Zhaokun Zhou, Yuesheng Zhu, Chao He, Yaowei Wang, Shuicheng Yan, Yonghong Tian, and Li Yuan. Spikformer: When spiking neural network meets transformer. In *International Conference on Learning Representations*, 2023.

Rui-Jie Zhu, Qiao Zhao, Guobin Zhang, Pengjie Deng, Hai Li, Yikang Duan, Xiang Cheng, et al. Spikegpt: Generative pre-trained language model with spiking neural networks. *arXiv preprint arXiv:2302.13939*, 2023.

A Code and Model Availability

- **Base Model**: Achieves the reported $r = 0.8031$ Pearson correlation on the STS benchmarks. (*URL omitted for double-blind review. Code and weights will be released upon acceptance.*)
- **Multilingual Model**: Extends the vocabulary to 64,000 to support cross-lingual semantic search across 20+ languages. (*URL omitted for double-blind review.*)